

PA2 实验报告

林晖鹏 2312966 计算机学院

目录

1. 概述
2. 实验目的
3. 实验内容
4. 阶段一：运行第一个 C 程序并实现基础指令
 - 4.1 指令执行过程
 - 4.2 从 `dummy` 开始定位未实现指令
 - 4.3 实现 `call`、`push`、`pop`、`sub`、`xor`、`ret`
 - 4.4 阶段一小结
5. 阶段二：补充更多指令、完善基础设施与 Differential Testing
 - 5.1 从 `dummy` 走向 `cputest`
 - 5.2 调试过程中遇到的典型问题
 - 5.3 Differential Testing
 - 5.4 一些对底层实现的理解
 - 5.5 Benchmark 与跑分验证
6. 阶段三：输入输出设备与运行更多程序
 - 6.1 串口与 `hello`
 - 6.2 时钟
 - 6.3 死循环检测
 - 6.4 键盘输入
 - 6.5 VGA 与 `_draw_rect()`
 - 6.6 运行 `typing`
 - 6.7 阶段三小结
7. 必答题
 - 7.1 `static inline` 为什么不能随便去掉
 - 7.2 在 `nemu` 目录下执行 `make` 后, `build/nemu` 是如何生成的
8. 思考
 - 8.1 一条指令在 NEMU 中的执行过程
 - 8.2 手册的重要性
9. 实验过程中遇到的问题与解决方法

- 9.1 `invalid opcode` 的第一反应不应该是猜，而是先去反汇编里定位机器码
- 9.2 `call rel32` 最容易写错的地方不是压栈，而是把相对偏移当成无符号数
- 9.3 `group` 指令最容易犯的错误不是语义写错，而是重复译码
- 9.4 `difftest` 的第一处差异出现在 `eflags`，并不一定意味着整条执行语义都错了
- 9.5 `volatile` 这个问题，只有看过优化前后的汇编，才真正能理解它为什么和设备寄存器有关
- 9.6 VGA 部分最容易出现“代码两边都看着像对了，但画面就是不出来”的情况

10. 总结

1. 概述

PA2 的主题是“简单复杂的机器：冯诺依曼计算机系统”。在 PA1 的基础上，PA2 进一步要求 NEMU 从最简单的 TRM 扩展为能够运行更多客户程序、具备运行时环境和输入输出能力的冯诺依曼计算机系统。本阶段涉及的内容明显比 PA1 更复杂，不仅要实现更多指令，还要理解取指、译码、执行、EFLAGS、Differential Testing、AM 抽象机器以及端口映射和内存映射 I/O。

2. 实验目的

- 理解 CPU 执行一条指令的完整流程。
- 通过补充 x86 指令，逐步跑通 `dummy` 和更多测试程序。
- 理解运行时环境、AM 和 Differential Testing 的作用。
- 实现串口、时钟、键盘、VGA 等设备相关功能。
- 进一步训练通过手册、反汇编和测试输出来定位问题的能力。

3. 实验内容

PA2 的主要任务可以分为三个阶段：

1. 跑通第一个 C 程序 `dummy`，实现第一批缺失指令。
2. 补充更多指令与基础设施，逐步通过 `cputest`、完善 `difftest` 和运行时环境。
3. 实现输入输出相关功能，最终运行 `hello`、`timetest`、`keytest`、`videotest`、`typing` 等程序。

4. 阶段一：运行第一个 C 程序并实现基础指令

4.1 指令执行过程

PA2 的第一项核心工作是理解一条指令在 NEMU 中是如何执行的。结合框架代码，可以把过程概括为：

1. 通过 `instr_fetch()` 从当前 `eip` 指向的内存中取出指令字节。
2. 以首字节作为 `opcode`，在 `opcode_table` 中进行查表。
3. 选择合适的 decode helper 和 exec helper。
4. 在译码阶段获得操作数信息。
5. 在执行阶段通过 RTL helper 完成指令语义。
6. 最后由 `update_eip()` 更新到下一条动态指令位置。

这一过程正对应了取指、译码、执行和更新程序计数器四个经典阶段。

4.2 从 `dummy` 开始定位未实现指令

在运行 `dummy` 时，NEMU 首先报出 `invalid opcode`。根据报错的 `eip` 位置，我到反汇编文件 `dummy-x86-nemu.txt` 中查找对应指令，再结合 i386 手册定位其具体含义。第一条未实现指令是：

- `call rel32`

继续向下分析后，`dummy` 这条执行路径还依赖：

- `push r32`
- `pop r32`
- `sub`
- `xor`
- `ret`

也就是说，要想真正跑通第一个 C 程序，不能只实现一个 `call`，而是要把这条函数调用路径上最基本的一批指令一起补齐。

下面这张图记录了第一次运行 `dummy` 时的报错现场，可以直接看到 `invalid opcode` 和对应的机器码位置。这也是后续顺着反汇编去定位 `call rel32` 的起点。

图 4-1 第一次运行 `dummy` 的报错现场

```
[src/monitor/monitor.c,69,load_img] The image is /home/linshangjin/code/ics2017/nexus-am/tests/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 20:26:16, Apr 22 2026
For help, type "help"
(nemu) c
invalid opcode(eip = 0x0010000a): e8 01 00 00 00 90 55 89 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x0010000a is not implemented.
2. Something is implemented incorrectly.
Find this eip(0x0010000a) in the disassembling result to distinguish which case it is.

If it is the first case, see

0306 Manual

for more details.

If it is the second case, remember:
* The machine is always right!
* Every line of untested code is always wrong!
```

4.3 实现 `call`、`push`、`pop`、`sub`、`xor`、`ret`

这一阶段完成了最先阻塞 `dummy` 的关键指令实现，并补上了若干必要的 RTL 基础能力。整体实现思路始终保持一致：

1. 根据测试失败位置定位缺失指令。
2. 查手册确认该指令的编码方式、操作数形式和标志位语义。
3. 在 `all-instr.h` 和 `opcode_table` 中补接线。
4. 在对应的执行文件中补 `exec helper`。
5. 如有需要，再回到底层 `rtl.h` 补 RTL helper。

其中，有几个关键点如下：

- `call rel32` 需要把返回地址压栈，并设置跳转目标。
- `push/pop` 依赖正确的栈顶移动顺序。
- `sub` 不仅要算出结果，还要正确更新 `CF/ZF/SF/OF`。
- `xor` 与 `sub` 对 `EFLAGS` 的影响并不相同。
- `ret` 本质上是从栈中弹出返回地址并恢复控制流。

和 PA1 相比，这里第一次明显感受到“补指令”不是只在某一个文件里加一段执行语义。以 `call rel32` 为例，实际涉及的改动链路包括：

- 在 `opcode_table` 中把 `0xe8` 正确接到 `IDEX(J, call)`；
- 在 `all-instr.h` 中声明 `exec_call`，否则 `exec.c` 构造表时编译不过；

- 在 `decode.c` 中补 `decode_op_SI()`，让有符号立即数能被正确读出；
- 在 `rtl.h` 中补 `rtl_push()`，为 `call` 和后面的 `push/ret` 提供统一的栈操作接口；
- 在 `control.c` 里实现 `exec_call()`，真正把返回地址压栈并设置跳转。

这一条链路中最关键的几段代码如下。

`rtl_push()` 的实现：

```
static inline void rtl_push(const rtlreg_t* src1) {
    rtl_subi(&cpu.esp, &cpu.esp, 4);
    rtl_sm(&cpu.esp, 4, src1);
}
```

这里先让 `esp -= 4`，再把 4 字节数据写到新的栈顶，符合 x86 栈向低地址增长的规则。如果把顺序写反，后面 `ret` 弹出的返回地址也会跟着出错。

`decode_op_SI()` 的核心逻辑：

```
uint32_t imm = instr_fetch(eip, op->width);
if (op->width == 1) {
    op->simm = (int8_t)imm;
} else {
    op->simm = (int32_t)imm;
}
```

这里补的是有符号立即数读取。`call rel32` 的 `rel32` 不是普通无符号常量，而是相对下一条指令地址的有符号偏移。如果这里直接把它当成无符号数，控制流目标就会错位。

`exec_call()` 的核心逻辑：

```
rtl_push(eip);
decoding.is_jump = 1;
```

这里没有直接去写 `cpu.eip`，而是沿用了 NEMU 当前的控制流框架：`decode_J` 先算出 `decoding.jump_eip`，执行阶段只负责说明“这是一条跳转指令”，最后由统一的 `update_eip()` 完成真正的 `eip` 更新。这样 `call`、`jmp`、`jcc` 这一类控制转移指令的处理风格就保持一致了。

补完 `call` 之后，`dummy` 并没有直接跑通，而是顺着函数序言继续暴露出 `push %ebp`、`sub $0x8, %esp`、`xor %eax, %eax`、`pop %ebp`、`ret` 这些基础指令缺失的问题。这也说明仅仅“补一条眼前报错的指令”是不够的，还要顺着当前这条真实执行路径，把函数调用这一整条最小链路一起打通。

这几条指令在原始实现里其实也分别处理过，而且它们之间是成套配合的，不是几条互相独立的补丁。

`push`、`pop` 和 `ret` 都直接依赖统一的栈操作接口。当前代码里：

```
static inline void rtl_push(const rtlreg_t* src1) {
    rtl_subi(&cpu.esp, &cpu.esp, 4);
    rtl_sm(&cpu.esp, 4, src1);
}

static inline void rtl_pop(rtlreg_t* dest) {
    rtl_lm(dest, &cpu.esp, 4);
    rtl_addi(&cpu.esp, &cpu.esp, 4);
}
```

在此基础上，`push` 和 `pop` 的执行 helper 比较直接：

```
make_EHelper(push) {
    rtl_push(&id_dest->val);
    print_asm_template1(push);
}

make_EHelper(pop) {
    rtl_pop(&t0);
    operand_write(id_dest, &t0);
    print_asm_template1(pop);
}
```

这里 `pop` 不能只写成“把值送回某个寄存器”，因为目标操作数也可能是内存，所以最终统一通过 `operand_write()` 回写。`ret` 则正好和 `call` 成对：

```
make_EHelper(ret) {
    rtl_pop(&decoding.jump_eip);
    decoding.is_jump = 1;
    print_asm("ret");
}
```

也就是说，`call` 压进去的是返回地址，`ret` 再把它弹出来交给统一的 `update_eip()`。从执行流上看，`call/push/pop/ret` 其实共享的是同一套栈语义。

`sub` 则是这一条函数序言里第一次真正要求更新标志位的算术指令。`dummy` 中对应的是：

```
83 ec 08    sub $0x8, %esp
```

这一条属于 `group1` 中的 `83 /5`，所以除了写 `exec_sub()` 之外，还必须在 `gp1` 表里把扩展 opcode `5` 接到 `sub`。当前实现的核心代码是：

```
rtl_sub(&t2, &id_dest->val, &id_src->val);
operand_write(id_dest, &t2);

rtl_update_ZFSF(&t2, id_dest->width);

rtl_sltu(&t0, &id_dest->val, &t2);
rtl_set_CF(&t0);

rtl_xor(&t0, &id_dest->val, &id_src->val);
rtl_xor(&t1, &id_dest->val, &t2);
rtl_and(&t0, &t0, &t1);
rtl_msb(&t0, &t0, id_dest->width);
rtl_set_OF(&t0);
```

这里真正关键的不是做减法，而是标志位更新。`rtl_update_ZFSF()` 负责结果相关的 `ZF/SF`，后面的 `rtl_sltu()` 和异或组合则分别在无符号和有符号语义下判断 `CF/OF`。如果这里只把结果写回寄存器而没有把标志位一起对齐，那么后面一旦进入 `cmp`、`jcc`、`setcc` 相关测试，就会很快暴露问题。

`xor` 则和 `sub` 正好形成对照。`dummy` 中出现的是：

```
31 c0    xor %eax, %eax
```

当前实现是：

```
rtl_xor(&t2, &id_dest->val, &id_src->val);
operand_write(id_dest, &t2);

rtl_update_ZFSF(&t2, id_dest->width);
```

```
rtl_set_CF(&tzero);
rtl_set_OF(&tzero);
```

这里的重点在于：逻辑运算和算术运算对标志位的影响不同。`xor` 会更新 `ZF/SF`，但会把 `CF/OF` 清零。因此 `xor %eax, %eax` 不只是“算出了 0”，而是同时把相关标志位也调整到符合逻辑运算语义的状态。这也是为什么它会成为一种常见的寄存器清零写法。

回头看这一小段执行路径，`call` 只是把控制流带进函数；真正让函数序言和函数返回完整跑起来的，是：

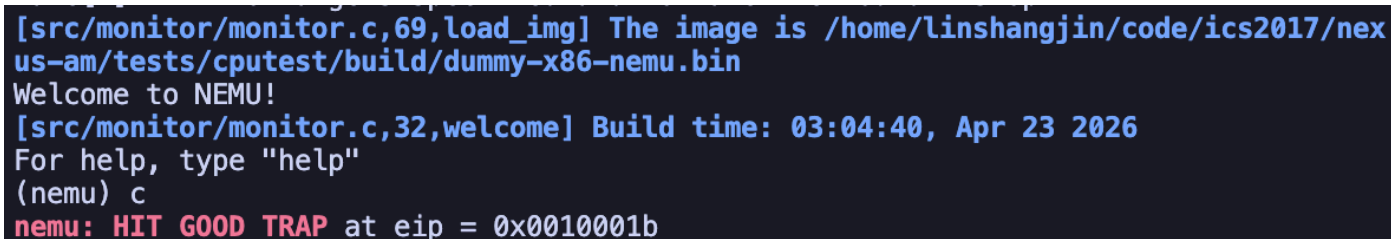
- `push / pop / ret` 这套栈操作链路；
- `sub` 这条会更新标志位的算术指令；
- `xor` 这条常见的逻辑清零指令。

也正因为这些实现是连在一起的，所以把它们作为“当前最小执行路径”的一组指令一起补齐，比单独盯住某一条报错指令更有效。

这一阶段完成后，`dummy` 可以顺利运行到 `HIT GOOD TRAP`，说明最基本的函数调用路径已经被打通。

下图对应的是补完第一批基础指令之后的运行结果。程序已经不再停在 `invalid opcode`，而是能够顺利执行到 `GOOD TRAP`，说明这一条最小函数调用链路已经跑通。

图 4-2 `dummy` 跑通结果



```
[src/monitor/monitor.c,69,load_img] The image is /home/linshangjin/code/ics2017/nexus-am/tests/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 03:04:40, Apr 23 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x0010001b
```

4.4 阶段一小结

做完阶段一，切身体会到：实现一条指令绝不是只写几行运算代码，而是要同时把“手册语义、译码方式、opcode 表接线、RTL helper、标志位更新”全部对齐。只要其中某一个环节出错，程序表面上看可能只是一个简单的 `invalid opcode`，但真实原因往往藏在更底层。

5. 阶段二：补充更多指令、完善基础设施与 Differential Testing

5.1 从 dummy 走向 cputest

在 dummy 跑通之后，继续依据 cputest 的失败项逐步补充指令和底层支持。最终这一阶段陆续实现和整理了如下内容：

- 数据传送与栈相关：mov、push、pop、leave、movsx、movzx、lea
- 算术相关：add、sub、cmp、inc、dec、neg、adc、sbb、mul、imul、div、idiv、cld、cwtl
- 逻辑与移位：test、and、or、xor、not、shl、shr、sar、setcc
- 控制转移：jmp、jcc、call、ret

与这些指令配套，还补上了若干十分关键的基础能力，例如：

- EFLAGS 位域与读写接口
- restart() 中 EFLAGS = 0x00000002 的初始化
- rtl_pop()、rtl_mv()、rtl_not()、rtl_sext()
- 若干判断、更新标志位和处理符号扩展的 RTL helper

这一阶段的特点是：已经不再是“哪条指令一眼就能看出来没写”，而是开始出现“指令语义写了，但某个细节和手册不一致”的问题。因此这一部分实际花时间最多的，反而是 EFLAGS、符号扩展、group 指令和 opcode 表接线。

例如 sub 虽然表面上只是减法，但真正补完整时至少要同时处理三件事：

1. 算出结果并写回目标操作数。
2. 根据结果更新 ZF/SF。
3. 按减法语义更新 CF/OF。

当前实现的核心代码如下：

```
rtl_sub(&t2, &id_dest->val, &id_src->val);
operand_write(id_dest, &t2);

rtl_update_ZFSF(&t2, id_dest->width);

rtl_sltu(&t0, &id_dest->val, &t2);
rtl_set_CF(&t0);

rtl_xor(&t0, &id_dest->val, &id_src->val);
rtl_xor(&t1, &id_dest->val, &t2);
rtl_and(&t0, &t0, &t1);
rtl_msb(&t0, &t0, id_dest->width);
rtl_set_OF(&t0);
```

这里可以看出，`sub` 真正麻烦的并不是减法本身，而是标志位更新规则。尤其是 `CF` 和 `OF` 的意义并不相同，前者对应无符号借位，后者对应有符号溢出。如果把这两个标志位混着写，后面 `cmp`、`jcc`、`setcc` 很快就会连带出错。

`xor` 则是另一类情况。它看似也会修改寄存器，但对标志位的影响和算术运算完全不同。当前实现里：

```
rtl_xor(&t2, &id_dest->val, &id_src->val);
operand_write(id_dest, &t2);

rtl_update_ZFSF(&t2, id_dest->width);
rtl_set_CF(&tzero);
rtl_set_OF(&tzero);
```

这对应了手册里的结论：逻辑指令会更新 `ZF/SF`，同时清掉 `CF/OF`。也正因为如此，`xor %eax, %eax` 才会成为常见的清零写法。

为了支撑这些算术和逻辑指令，这一阶段还把 `EFLAGS` 真正接进了 `CPU_state`，并在 `restart()` 中补了：

```
cpu.eip = ENTRY_START;
cpu.eflags = 0x00000002;
```

这一步看起来只是一个初始化赋值，但它实际上把“实验里临时要用的几个标志位”和“i386 手册中处理器复位后的初始状态”真正对应起来了。后面开启 `difftest` 时，`eflags` 的初始值是否正确会直接体现在第一条指令的比较结果上。

这部分在当前代码中也能直接看到。`reg.h` 里把 `EFLAGS` 放进了 `CPU_state`：

```
union {
    uint32_t eflags;
    struct {
        uint32_t CF : 1;
        uint32_t   : 5;
        uint32_t ZF : 1;
        uint32_t SF : 1;
        uint32_t TF : 1;
        uint32_t IF : 1;
        uint32_t DF : 1;
        uint32_t OF : 1;
        ...
    };
};
```

而 `rtl.h` 里又通过一组统一接口来读写这些位：

```
#define make_rtl_setget_eflags(f) \
    static inline void concat(rtl_set_, f) (const rtlreg_t* src) { \
        cpu.f = *src & 0x1; \
    } \
    static inline void concat(rtl_get_, f) (rtlreg_t* dest) { \
        *dest = cpu.f; \
    }

make_rtl_setget_eflags(CF)
make_rtl_setget_eflags(OF)
make_rtl_setget_eflags(ZF)
make_rtl_setget_eflags(SF)
```

这样的好处是，后面在执行 helper 里更新标志位时，不需要直接到处手写位运算，而是统一通过 `rtl_set_CF()`、`rtl_get_CF()` 这一类 helper 完成。也就是说，PA2 这里不是简单把几个标志位“塞进结构体里”，而是把它们真正接入了 RTL 层。

除了 `sub` 和 `xor` 这种已经在 `dummy` 里直接出现的指令，这一阶段继续往 `cputest` 推进时，还逐步补上了更多“表面简单、实际上很容易踩手册细节”的实现。

例如数据传送和栈相关指令里，`push`、`pop`、`leave` 的实现都比较直接，但它们各自对应的语义边界并不完全一样。当前 `data-mov.c` 里的实现是：

```
make_EHelper(push) {
    rtl_push(&id_dest->val);
    print_asm_template1(push);
}

make_EHelper(pop) {
    rtl_pop(&t0);
    operand_write(id_dest, &t0);
    print_asm_template1(pop);
}

make_EHelper(leave) {
    rtl_mv(&cpu.esp, &cpu.ebp);
    rtl_pop(&cpu.ebp);
    print_asm("leave");
}
```

这里 `leave` 最值得单独说明。它并不是一条全新的“复杂指令”，而是函数尾声的一种快捷写法，本质等价于：

```
mov esp, ebp
pop ebp
```

也正因为如此，只要 `rtl_mv()` 和 `rtl_pop()` 已经正确实现，`leave` 本身的执行 helper 反而很短。这个实现过程也让我更清楚地看到：很多 x86 指令在硬件层面看似独立，在语义层面其实可以拆解成几步更基础的寄存器和栈操作。

再比如 `push imm8`，这条指令的真正难点不在 `push`，而在 `imm8` 的符号扩展。当前原始实现里是通过 `push_SI + decode_op_SI()` 这一组组合来处理的，也就是说，关键不在执行 helper，而是在译码阶段先把 8 位立即数变成正确的有符号值：

```
uint32_t imm = instr_fetch(eip, op->width);
if (op->width == 1) {
    op->simm = (int8_t)imm;
}
else {
    op->simm = (int32_t)imm;
}
rtl_li(&op->val, op->simm);
```

这样一来，`push imm8` 在进入执行阶段之前，就已经不再是“一个原始的 8 位机器码”，而是一个按手册规则符号扩展后的操作数。也正因为这个逻辑是在译码阶段完成的，所以后面的执行 helper 可以继续复用普通的 `push` 流程，不需要为了 `imm8` 再单独发明一套执行语义。

算术和乘除法这一边，除了 `sub/cmp/inc/dec/neg` 这些基础指令之外，原始实现里 `imul/div/idiv` 的处理也比表面看起来复杂。当前 `arith.c` 里的 `imul2` 会先做符号扩展，再调用统一的 RTL 乘法接口：

```
make_EHelper(imul2) {
    rtl_sext(&id_src->val, &id_src->val, id_src->width);
    rtl_sext(&id_dest->val, &id_dest->val, id_dest->width);

    rtl_imul(&t0, &t1, &id_dest->val, &id_src->val);
    operand_write(id_dest, &t1);

    print_asm_template2(imul);
}
```

这里最容易掉坑的地方反而不是乘法本身，而是：

- 双操作数形式和单操作数形式是不是接到了同一个 helper；

- 操作数在进入 RTL 之前有没有先做符号扩展；
- `0f af` 这种两字节 opcode 有没有放到 2-byte opcode table 中。

这也是为什么原始调试记录里会专门提到：有些问题不是 helper 没写，而是明明 helper 已经存在，结果却因为 opcode 表接错了地方而仍然报 `invalid opcode`。

逻辑与移位这一组同样如此。`logic.c` 里当前的实现分工其实很清晰：

```
make_EHelper(test) {
    rtl_and(&t2, &id_dest->val, &id_src->val);
    rtl_update_ZFSF(&t2, id_dest->width);
    rtl_set_CF(&tzero);
    rtl_set_OF(&tzero);
}

make_EHelper(and) { ... operand_write(...); ... }
make_EHelper(or)  { ... operand_write(...); ... }
make_EHelper(xor) { ... operand_write(...); ... }
make_EHelper(not) { rtl_mv(&t2, &id_dest->val); rtl_not(&t2); ... }
```

这里可以清楚看到：

- `test` 像 `and`，但只更新标志位，不写回结果；
- `and/or/xor` 都会写回结果，同时清 `CF/OF`；
- `not` 只改操作数本身，不改标志位。

这几条指令看起来都属于“逻辑运算”，但真正实现时手册对标志位的要求并不相同。如果只从运算符表面去写，很容易把它们混成同一类模板。

最后，原始实现里其实还很强调一件事：这一阶段很多 bug 不在某条 helper，而在 group 指令和 opcode 表接线。当前 `exec.c` 里最关键的两段就是：

```
make_group(gp1,
    EX(add), EX(or), EX(adc), EX(sbb),
    EX(and), EX(sub), EX(xor), EX(cmp))

make_group(gp2,
    EX(rol), EMPTY, EMPTY, EMPTY,
    EX(shl), EX(shr), EMPTY, EX(sar))
```

以及：

```
/* 0x68 */ IDEX(push_SI, push), EMPTY, IDEXW(push_SI, push, 1), EMPTY,
/* 0x80 */ IDEXW(I2E, gp1, 1), IDEX(I2E, gp1), EMPTY, IDEX(SI2E, gp1),
```

```
/* 0xac */ EMPTY, EMPTY, EMPTY, IDEX(E2G, imul2),
```

这几行很能说明 PA2 后半段为什么越来越容易出现“语义写了，但还是跑不过”的情况：

- `push imm8` 是否接到了 `0x6a`；
- `83 /5` 是否通过 `gp1` 扩展到了 `sub`；
- `0f af` 这种两字节 opcode 是否接到了后半张 opcode 表；
- group 表内部是不是误用了 `IDEX(...)` 导致重复译码。

所以从 `dummy` 进入 `cputest` 之后，这部分真正花时间的地方其实不只是“再补几条新指令”，而是不断检查：当前这条指令的语义、译码方式、宽度、符号扩展和 opcode 接线，是否真的全都和手册与框架一致。

5.2 调试过程中遇到的典型问题

这一阶段真正耗时的地方往往不是“不会写某条指令”，而是一些更隐蔽的细节错误，例如：

- opcode 表接错槽位
- 两字节 opcode 错接到单字节表
- group 指令重复译码
- 立即数宽度或符号扩展处理错误
- 标志位更新和手册不一致

这些问题说明，PA2 里“实现一条指令”的难点并不只在执行语义本身，而在于能否把手册、反汇编、译码框架和 RTL 行为统一起来。

5.3 Differential Testing

PA2 中的 Differential Testing 让我第一次明确感受到“参考实现”的价值。它的核心思想不是只看程序有没有跑起来，而是将 NEMU 当前执行到某一条指令后的寄存器状态，与参考实现进行逐条比较。一旦出现不一致，就说明当前实现某处行为偏离了正确语义。

这使得许多问题可以在很早的阶段暴露出来，而不必等到程序跑飞之后再从头倒查。尤其是在实现 EFLAGS、条件跳转和多种算术逻辑指令时，`difftest` 对定位错误非常有帮助。

下图展示的是开启 `difftest` 之后首次出现寄存器状态不一致时的现场。它的作用不是简单地说明“程序错了”，而是帮助把排查范围迅速收缩到 `eflags` 相关逻辑。

图 5-1 diffest 首次发现状态差异

```
[src/monitor/diff-test/diff-test.c,96,init_diffest] Connect to QEMU successfully
[src/monitor/monitor.c,69,load_img] The image is /home/linshangjin/code/ics2017/nexus-am/tests/cputest/build/add-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 06:56:12, Apr 23 2026
For help, type "help"
(nemu) c
Diffest mismatch at eip = 0x00100000: eflags, NEMU = 0x00000002, QEMU = 0x00000006
```

这一阶段一个很直观的例子就是 **eflags** 的第一处 diff。最开始直接比较整个位图时，NEMU 和 QEMU 在第一条指令后就出现：

```
Diffest mismatch at eip = 0x00100000:
eflags, NEMU = 0x00000002, QEMU = 0x00000006
```

继续看差值后可以发现，多出来的是 **PF**。但当前 PA2 实现里真正关心并已经接入的只有 **CF/ZF/SF/IF/OF**，所以这里并不是执行语义彻底错了，而是比较策略过宽。后面把比较改成带 mask 的形式，只保留当前已经实现的那些位之后，diffest 才重新回到“暴露真实行为错误”这个用途上。

核心写法类似：

```
const uint32_t eflags_mask =
    (1u << 0) |
    (1u << 6) |
    (1u << 7) |
    (1u << 9) |
    (1u << 11);

if ((cpu.eflags & eflags_mask) != (r.eflags & eflags_mask)) {
    diff = true;
}
```

另外，这一阶段还明显感受到 **opcode_table** 的接线密度已经远高于阶段一。当前 **exec.c** 里最关键的片段包括：

```
make_group(gp1,
    EX(add), EX(or), EX(adc), EX(sbb),
    EX(and), EX(sub), EX(xor), EX(cmp))

make_group(gp2,
    EX(rol), EMPTY, EMPTY, EMPTY,
    EX(shl), EX(shr), EMPTY, EX(sar))
```

以及：

```
/* 0xe8 */ IDEX(J, call), IDEX(J, jmp), EMPTY, IDEXW(J, jmp, 1),
```

和两字节 opcode 表中的：

```
/* 0x80 */ IDEX(J, jcc), ...  
/* 0x90 */ IDEXW(E, setcc, 1), ...  
/* 0xac */ EMPTY, EMPTY, EMPTY, IDEX(E2G, imul2),  
/* 0xb6 */ IDEXW(E2G, movzx, 1), ...  
/* 0xbe */ IDEXW(E2G, movsx, 1), ...
```

这几段代码特别能说明 PA2 中“补指令”的真实工作量。很多时候 helper 本身并不复杂，真正容易出错的是：

- 它应该接在单字节表还是两字节表；
- 它应该落在普通 opcode 还是 group 扩展表；
- group 表里应该直接 `EX(...)` 还是外层还需要 `IDEX(...)`。

也正因为这个原因，PA2 后半段很多 bug 表面上像“某条指令执行错了”，实际上根源常常是 opcode 接错了位置。

5.4 一些对底层实现的理解

在完成这一阶段时，对 `rtl.h` 里的 `static inline`、Makefile 的组织方式以及“编译”和“链接”的区别也有了更清晰的认识。`rtl.h` 中的小型 helper 写成 `static inline`，既能避免头文件定义在多个编译单元中引发链接冲突，又能减少生成大量局部函数实体带来的冗余。Makefile 则通过模式规则把 `src/*.c` 编译为 `build/obj/*.o`，最后再统一链接成 `build/nemu`。

这些内容虽然不属于“某条指令”的直接实现，但它们决定了整个工程是如何被组织、编译和运行起来的。

例如 `rtl.h` 中的 helper 之所以大量写成：

```
static inline void rtl_li(rtlreg_t* dest, uint32_t imm) {  
    *dest = imm;  
}
```

并不是风格问题，而是头文件函数定义在 C 工程里的一个典型约束：

- 如果去掉 `static`，这些函数就会变成外部链接符号，但工程里又没有在某个 `.c` 文件中单独提供一份全局定义，最终容易在链接阶段出现 `undefined reference`；
- 如果去掉 `inline`，每个包含 `rtl.h` 的编译单元都会生成一份本地函数实体，而 NEMU 又开启了 `-Wall -Werror`，很多未被使用的 helper 很容易触发 `defined but not used` 一类告警；
- 如果 `static` 和 `inline` 都去掉，那么头文件里的定义会被复制进多个编译单元，最终又会在链接阶段变成 `multiple definition`。

这也是为什么 PA2 后面的很多 RTL helper 虽然看起来都很短，但仍然统一写成 `static inline`。它们本质上更接近“让执行 helper 在语义层调用的一套伪指令”，而不是工程里真正独立导出的公共函数。

Makefile 这一边也是同样的道理。当前 `nemu/Makefile` 里最关键的几行是：

```
SRCS = $(shell find src/ -name "*.c")
OBJS = $(SRCS:src/%.c=$(OBJ_DIR)/%.o)

$(OBJ_DIR)/%.o: src/%.c
    @$(CC) $(CFLAGS) -c -o $@ $<
```

这里实际上把整个构建过程拆成了两个很清晰的阶段：

1. 枚举 `src/` 下所有 `.c` 文件，并映射到 `build/obj/` 下对应的 `.o`；
2. 等所有 `.o` 准备好之后，再统一链接成 `build/nemu`。

再结合：

```
-include $(OBJS:.o=.d)
```

可以看出 `-MMD` 生成的依赖文件也被接回了 Makefile 本身。因此如果某个头文件改了，`make` 也能自动知道哪些目标文件需要重编译。这一套机制虽然在 PA2 里不像 `call`、`sub` 那样直接体现在指令语义上，但它决定了每次补完指令之后，工程是如何被稳定地重新构建出来的。

5.5 Benchmark 与跑分验证

在 `cputest` 基本跑通之后，又继续运行了 `dhrystone`、`coremark` 和 `microbench`。这一步主要是检查当前实现能否继续支撑规模更大、覆盖面更宽的程

序。

其中 `microbench` 在继续推进时暴露出了一个新的缺口。`md5` 子项执行到：

```
d3 c0
```

时触发了 `invalid opcode`。继续沿着机器码和手册往下查，可以确认这对应的是：

```
rol eax, cl
```

这说明问题不是已有指令实现错误，而是 `benchmark` 确实继续覆盖到了此前还没补上的旋转类指令。这里补了三个位置：`rtl.h` 里的 `rtl_rol()`、`logic.c` 里的 `make_EHelper(rol)`，以及 `exec.c` 里 `gp2` 分发表对 `/0` 的接线。关键代码如下：

```
static inline void rtl_rol(rtlreg_t* dest, const rtlreg_t* src1,
    const rtlreg_t* src2, int width) {
    uint32_t bits = width * 8;
    uint32_t count = *src2 & 0x1f;
    uint32_t value = *src1;
    uint32_t mask = (width == 4 ? 0xfffffffffu : ((1u << bits) - 1));

    value &= mask;
    count %= bits;

    if (count == 0) {
        *dest = value;
        return;
    }

    *dest = ((value << count) | (value >> (bits - count))) & mask;
}
```

执行 helper 如下：

```
make_EHelper(rol) {
    rtl_rol(&t2, &id_dest->val, &id_src->val, id_dest->width);
    operand_write(id_dest, &t2);
    print_asm_template2(rol);
}
```

在 `gp2` 分组表中，还需要把 `/0` 接到 `rol`：

```
make_group(gp2,
    EX(rol), EMPTY, EMPTY, EMPTY,
    EX(shl), EX(shr), EMPTY, EX(sar))
```

补完这部分之后，**microbench** 才能够继续向下运行。

这三项 benchmark 的实验记录如下：

```
Dhrystone PASS          52 Marks
                        vs. 100000 Marks (i7-6700 @ 3.40GHz)
```

```
CoreMark PASS          180 Marks
                        vs. 100000 Marks (i7-6700 @ 3.40GHz)
```

```
[md5] MD5 digest: * Passed.
MicroBench PASS        207 Marks
                        vs. 100000 Marks (i7-6700 @ 3.40GHz)
nemu: HIT GOOD TRAP at eip = 0x00100032
```

从这组结果来看，**dhrystone**、**coremark** 和 **microbench** 都已经能够在当前 NEMU 上完整运行；同时 **microbench** 在推进过程中还额外暴露出了 **rol** 这样的缺失指令。第二阶段的工作到这里已经不只是把 **dummy** 和 **cputest** 跑通，而是能够继续支撑 benchmark 这类规模更大、覆盖面更宽的测试程序。

为了更直观看看运行环境对结果的影响，又和两位同学的结果做了一个对比。其中一位同学同样是在虚拟机环境中完成实验，另一位同学则是在本地 **docker** 环境中运行。整理后的结果如下：

环境	Dhrystone	CoreMark	MicroBench
本次实验（虚拟机）	52	180	207
同学 A（虚拟机）	45	151	163
同学 B（本地 Docker）	281	911	1101

从这个对比可以看出，虚拟机环境下的两组结果整体比较接近，而本地 **docker** 环境的三项分数都明显更高。这一差异基本符合预期，因为 NEMU 本身就是在宿主机上用软件模拟客户机指令执行；如果外层再套一层虚拟机，那么客户程序执行一次指令，实际上

要经过更长的一条执行链路，额外的指令解释、调度和图形界面开销都会继续放大运行时间。

因此，这里的跑分更适合用来做同一环境下的阶段性对比，而不太适合直接拿不同机器、不同运行环境的绝对数值做横向评价。就本次实验而言，benchmark 已经能够稳定跑完，三项程序都得到有效结果，这一点比具体分数高低更能说明当前实现已经具备了继续运行复杂程序的能力。

6. 阶段三：输入输出设备与运行更多程序

6.1 串口与 hello

在设备部分，串口输出是最早打通的内容之一。串口能够工作之后，运行简单程序时就可以看到程序输出，这对于验证后续系统功能是否正确非常重要。

从实现角度看，这一步并不是单纯“让 **hello** 打印出一行字”，而是把 AM/NEMU 之间的端口输入输出接口真正接了起来。接上后，当前 **system.c** 里的 **in/out** helper 是：

```
make_EHelper(in) {
    t0 = pio_read(id_src->val, id_dest->width);
    operand_write(id_dest, &t0);
    print_asm_template2(in);
}

make_EHelper(out) {
    pio_write(id_dest->val, id_src->width, id_src->val);
    print_asm_template2(out);
}
```

这里 **in** 的语义是“从端口读数据到累加器”，**out** 的语义是“把累加器里的数据写到端口”。它们看起来很短，但正是这两个 helper，把客户程序里针对 I/O 端口的访问转成了 NEMU 设备模型中的 **pio_read/pio_write**。如果这一步没有接上，那么即使设备模块本身存在，上层程序也还是无法真正和设备交互。

也就是说，阶段三里的设备功能并不是孤立工作的，它首先依赖 CPU 能把 **in/out** 这样的 I/O 指令正确执行出来。

下图对应的是串口链路打通之后运行简单程序的输出结果。它说明 AM 到 NEMU 的端口 I/O 这一条最基础的设备输出路径已经可用。

图 6-1 串口输出 *hello* 结果

```
[src/monitor/monitor.c,69,load_img] The image is /home/linshangjin/code/ics2017/nexus-am/apps/hello/build/hello-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 10:39:40, Apr 23 2026
For help, type "help"
(nemu) c
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
nemu: HIT GOOD TRAP at eip = 0x0010006e
```

6.2 时钟

在实现时钟相关功能时，这里补充了时间读取接口，并进一步理解了运行时环境中“当前时间”是如何由底层设备模拟出来的。

时钟这一部分在 NEMU 端对应的核心代码位于 *timer.c*。当前实现里，RTC 设备通过端口 *0x48* 暴露一个“当前毫秒数”寄存器：

```
#define RTC_PORT 0x48

void init_timer() {
    rtc_port_base = add_pio_map(RTC_PORT, 4, rtc_io_handler);
}
```

这一步的作用是把端口 *0x48* 注册成一个 PIO 设备。之后只要客户程序执行 *in* 指令访问这个端口，最终就会进入 *rtc_io_handler()*。

真正读取时间时，设备回调会在 *rtc_io_handler()* 中更新寄存器内容：

```
struct timeval now;
gettimeofday(&now, NULL);
uint32_t ms = seconds * 1000 + (useconds + 500) / 1000;
if (ms <= rtc_last_ms) {
    ms = rtc_last_ms + 1;
}
rtc_last_ms = ms;
rtc_port_base[0] = ms;
```

这里有一个很值得记录的细节：当前实现没有简单地直接返回 `gettimeofday()` 换算出的毫秒值，而是额外做了一次“单调递增”修正。原因是某些 benchmark 会用两次读出的时间差去做除法，如果两次读取恰好落在同一毫秒内，就可能得到 `0ms`，进一步触发除零。这种问题如果不看设备实现本身，只从上层测试现象看，是很难一下子想到会和时钟粒度有关的。

AM 端 `_uptime()` 的实现则比较直接：

```
unsigned long now = inl(RTC_PORT);  
return now - boot_time;
```

也就是把设备暴露出来的绝对毫秒值，转成“系统自启动以来经过了多久”。这里的 `boot_time` 会在 `_ioe_init()` 时先读一次 RTC 记录下来，因此后续 `_uptime()` 返回的是相对时间而不是绝对时间戳。这样 `timetest` 和后面的 benchmark 才能使用统一的时间接口，而不需要直接关心底层设备是如何模拟出来的。

从整条链路看，这一部分实际串起来的是：

1. CPU 正确执行 `in` 指令；
2. `system.c` 中的 `pio_read()` 把访问转发给对应端口；
3. `timer.c` 在回调里准备当前毫秒值；
4. AM 端 `_uptime()` 再把这个值转换成“开机以来经过了多久”。

也正因为这四步是串在一起的，所以计时功能一旦异常，不一定只是设备本身的问题，也可能和 `in` 指令、PIO 接线或 AM 接口实现有关。

下图展示的是时钟接口已经工作后的测试结果，可以直接看到程序已经能够读取并输出运行时间信息。

图 6-2 时钟测试结果

```
1 second.  
2 seconds.  
3 seconds.  
4 seconds.  
5 seconds.  
6 seconds.  
7 seconds.  
8 seconds.  
9 seconds.  
10 seconds.  
11 seconds.  
12 seconds.  
13 seconds.  
14 seconds.  
15 seconds.  
16 seconds.  
17 seconds.  
18 seconds.  
19 seconds.  
20 seconds.
```

6.3 死循环检测

除了时钟本身，还在 `cpu-exec.c` 里加了一个用于调试的**死循环检测**逻辑。这部分功能不属于讲义要求必须完成的 CPU 指令或设备语义，但在真实调试过程中非常有价值，因为很多时候程序卡住并不会立刻报出 `invalid opcode` 或 `assertion`，而是长时间停留在少数几条指令之间反复跳转。

当前这部分逻辑在 `nemu/src/monitor/cpu-exec.c` 中，相关宏定义如下：

```
#define ENABLE_LOOP_DETECTOR 0  
#define LOOP_WINDOW_SIZE 32  
#define LOOP_REPEAT_THRESHOLD 20
```

这里的思路不是去严格证明“程序一定死循环”，而是做一个保守的启发式判断。整体流程可以分成四步：

1. 每执行完一条指令，就把当前 `eip` 记录进一个固定大小的滑动窗口。
2. 当窗口被填满之后，统计窗口里不同 `eip` 的个数。
3. 如果窗口里一直只有很少几个不同地址，说明程序可能只是在一个很小的循环体里打转。
4. 如果这种情况连续出现很多轮，就给出“可能陷入死循环”的提示，并把执行停回 `monitor`。

当前用于维护窗口状态的变量是：

```
static vaddr_t loop_window[LOOP_WINDOW_SIZE];
static int loop_window_pos = 0;
static int loop_window_count = 0;
static uint64_t loop_repeat_count = 0;
```

其中：

- `loop_window` 用来保存最近一段时间执行到的 `eip`
- `loop_window_pos` 表示下一次要覆盖的位置
- `loop_window_count` 表示窗口当前已经装了多少条记录
- `loop_repeat_count` 用来记录“疑似小范围循环”的情况已经连续出现了多少轮

真正把新 `eip` 放进窗口的代码是：

```
loop_window[loop_window_pos] = eip;
loop_window_pos = (loop_window_pos + 1) % LOOP_WINDOW_SIZE;
if (loop_window_count < LOOP_WINDOW_SIZE) {
    loop_window_count++;
    return false;
}
```

这一段先不断填满窗口。在窗口还没有装满之前，不做后续判断，而是直接返回 `false`。这样做是合理的，因为样本太少时根本看不出执行轨迹是否真的陷入了很小的循环。

窗口装满之后，接下来会统计其中有多少个不同的 `eip`：

```
for (i = 0; i < LOOP_WINDOW_SIZE; i++) {
    int j;
    seen = false;
    for (j = 0; j < i; j++) {
        if (loop_window[i] == loop_window[j]) {
            seen = true;
            break;
        }
    }
    if (!seen) {
        unique++;
    }
}
```

这里的 `unique` 可以理解成：最近 32 次执行里，程序实际上只在多少个不同的地址之间跳转。如果这个值非常小，就说明执行轨迹高度集中，很像在少数几条指令之间来回

打转。

最终的判定逻辑是：

```
if (unique <= 8) {
    loop_repeat_count++;
}
else {
    loop_repeat_count = 0;
}

if (loop_repeat_count >= LOOP_REPEAT_THRESHOLD) {
    printf("Possible infinite loop detected around eip = 0x%08x.\n", eip);
    printf("Execution has been paused automatically.\n");
    return true;
}
```

这里有两个关键阈值：

- `unique <= 8` 表示最近一个窗口里的执行地址种类非常少；
- `loop_repeat_count >= LOOP_REPEAT_THRESHOLD` 表示这种高度重复的模式已经连续出现很多轮，而不是偶然的一小段局部循环。

也就是说，这里不会因为程序短时间在几条指令之间跳几次就立刻停下来，而是要求这种特征持续一段时间之后才触发提示。这样做的目的是尽量避免把正常的等待循环、初始化阶段的小循环或者某些紧凑控制流误判为真正的死循环。

这一逻辑最终是在 `cpu_exec()` 中接入的：

```
if (detect_possible_infinite_loop(cpu.eip)) {
    nemu_state = NEMU_STOP;
}
```

因此每执行完一条指令，NEMU 都会顺手检查一次最近的执行轨迹是否符合“长时间局限在少数几个 `eip` 中”的模式。一旦命中条件，就主动停回 monitor，并打印：

```
Possible infinite loop detected around eip = ...
Execution has been paused automatically.
```

这部分功能更像是调试辅助，而不是体系结构语义的一部分，但它很适合拿来定位 benchmark 卡住、程序逻辑异常或者某些一直不退出的自旋场景。尤其是在阶段三之

后，程序已经开始和时钟、键盘、图形设备交互，单纯靠 `invalid opcode` 这类错误已经不足以覆盖所有问题，这种执行轨迹层面的辅助检测就会变得更有价值。

下图给出了启用这项辅助检测后，程序被自动暂停时的界面。它对应的不是某条特定指令语义错误，而是执行轨迹已经呈现出明显的小范围重复特征。

图 6-3 死循环检测触发结果

```
[src/monitor/monitor.c,69,load_img] The image is /home/linshangjin/code/ics2017/nexus-am/tests/cputest/build/loop-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 07:04:41, Apr 23 2026
For help, type "help"
(nemu) c
Possible infinite loop detected around eip = 0x00100030.
Execution has been paused automatically.
(nemu) █
```

6.4 键盘输入

根据讲义和 NEMU 键盘设备实现，键盘输入主要依赖两个端口：

- `0x64`：状态寄存器
- `0x60`：数据寄存器

因此 `_read_key()` 的思路很明确：先读取状态寄存器判断是否有按键，再从数据寄存器取回完整键码。完成后，`keytest` 等上层程序才能真正读到按键信息。

这一部分虽然代码量不大，但它把“设备已经在 NEMU 端存在”和“应用程序真的能读到按键”这两层真正接通了。最开始检查 `ioe.c` 时，`_read_key()` 仍然是一个直接返回 `_KEY_NONE` 的空实现，这种情况下即使 NEMU 的 `keyboard.c` 已经在往队列里塞事件，上层程序看到的也始终是“没有输入”。

补完之后，逻辑基本就是：

```
#define I8042_DATA_PORT 0x60
#define I8042_STATUS_PORT 0x64
#define I8042_STATUS_HASKEY_MASK 0x1
```

先通过 `0x64` 读取状态寄存器判断当前是否有键，再在有键的情况下从 `0x60` 取出完整键码。这样 `keytest` 中看到的按下和松开事件，才会真正对应到底层设备模型。

这一部分当前实现里的关键代码是：

```
int _read_key() {
    if ((inb(I8042_STATUS_PORT) & I8042_STATUS_HASKEY_MASK) == 0) {
```

```

    return _KEY_NONE;
}
return inl(I8042_DATA_PORT);
}

```

这里的结构很重要。先读状态寄存器，是为了避免在“当前没有键”的时候误读数据端口；只有确认最低位表示当前确实有键时，才去读 `0x60`。从 AM 的角度看，这一步就是把 NEMU 端已经排队好的键盘事件安全地暴露给上层接口。

从“为什么这样写”这个角度看，`_read_key()` 其实体现了 IOE 的一个基本模式：AM 并不自己维护设备状态，它只负责用端口访问的方式，把 NEMU 端已经实现好的设备模型包装成统一接口。如果把这层关系看清楚，PA2 后面很多设备代码就不容易写偏。

再往 NEMU 端看，键盘设备本身在 `keyboard.c` 中还有一层完整的缓冲和状态维护。当前实现里：

```

#define KEY_QUEUE_LEN 1024
static int key_queue[KEY_QUEUE_LEN];
static int key_f = 0, key_r = 0;

```

`send_key()` 会把 SDL 产生的按键事件转换成 AM 键码并塞入队列：

```

uint32_t am_scancode = keymap[scancode] | (is_keydown ? KEYDOWN_MASK : 0);
key_queue[key_r] = am_scancode;
key_r = (key_r + 1) % KEY_QUEUE_LEN;

```

而 `i8042_io_handler()` 则负责把队列里的事件暴露到两个端口寄存器上：

```

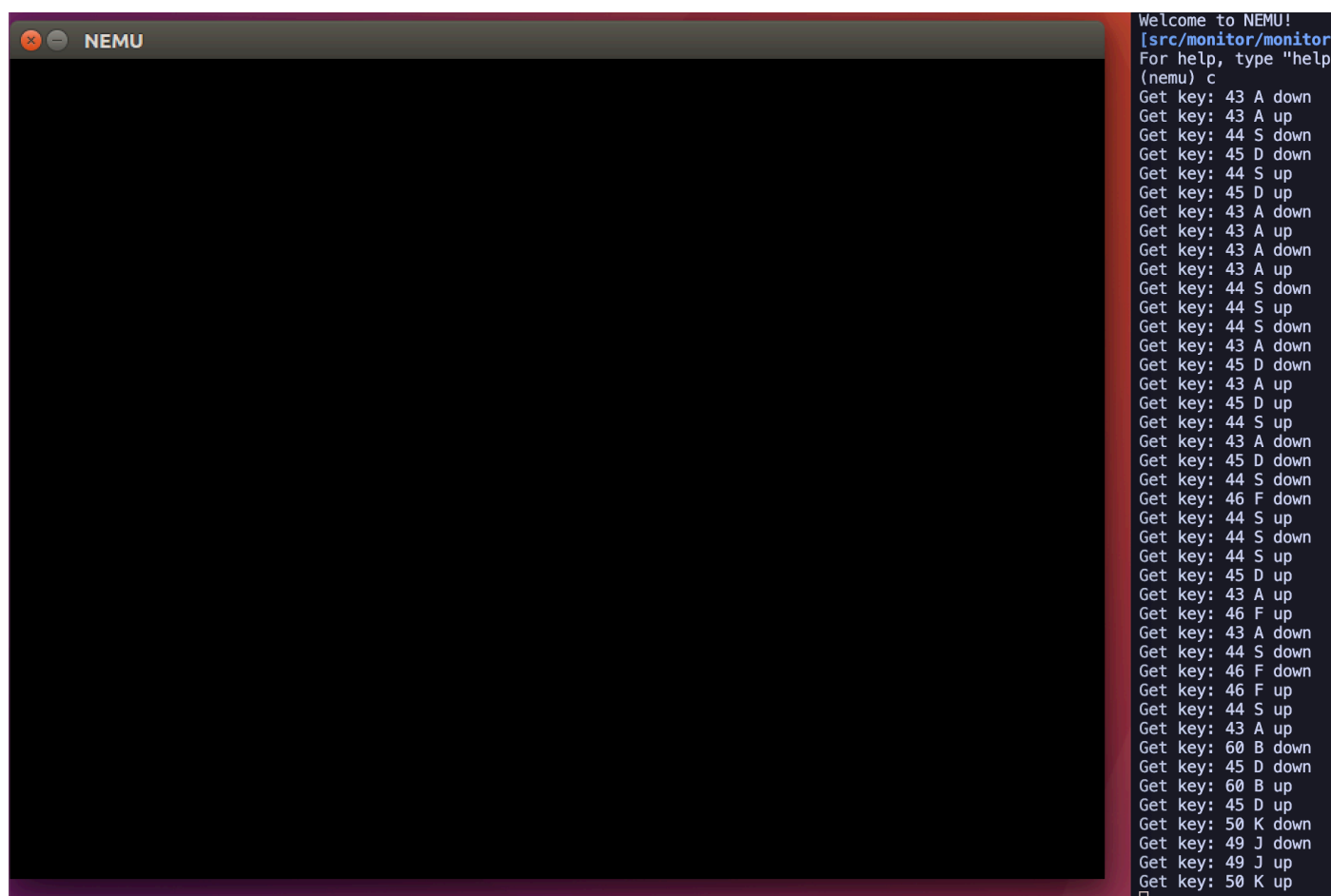
if (addr == I8042_DATA_PORT) {
    i8042_status_port_base[0] &= ~I8042_STATUS_HASKEY_MASK;
}
else if (addr == I8042_STATUS_PORT) {
    if ((i8042_status_port_base[0] & I8042_STATUS_HASKEY_MASK) == 0) {
        if (key_f != key_r) {
            i8042_data_port_base[0] = key_queue[key_f];
            i8042_status_port_base[0] |= I8042_STATUS_HASKEY_MASK;
            key_f = (key_f + 1) % KEY_QUEUE_LEN;
        }
    }
}
}

```

这段代码特别能说明为什么 `_read_key()` 不能一上来就盲读数据寄存器。NEMU 端其实是通过“状态位 + 数据寄存器”这组组合来模拟真实键盘控制器的行为：先告诉上层“现在有没有键”，再把一条键盘事件交出去。AM 端只有按这个协议访问，整个输入链路才能稳定工作。

下图展示的是 `keytest` 运行时读到的按键事件。可以看到按下和松开都已经能够被正确识别出来，这说明键盘输入链路已经真正打通。

图 6-4 键盘测试结果



6.5 VGA 与 `_draw_rect()`

进入 VGA 部分后，需要同时在两边补齐逻辑：

- NEMU 端：在物理内存读写中接入 MMIO 分流，使显存映射真正生效。
- AM 端：把 `_draw_rect()` 从占位实现改成按矩形逐行写入 framebuffer 的正确实现。

只有这两部分同时正确，`videotest` 和后续的 `typing` 才能正常显示图像。为了让实现更加稳妥，我还在 `_draw_rect()` 中加入了边界裁剪，避免矩形越界破坏后续内存。

这一部分最容易出的问题是“只改通了一半”。一开始如果只看 `vga.c`，会觉得显存映射已经存在；但继续顺着调用链往下看，就会发现还差两头：

- NEMU 端的 `paddr_read()` / `paddr_write()` 还没有把访问分流到 MMIO；
- AM 端的 `_draw_rect()` 也还是占位版本，没有真正按矩形逐行写 framebuffer。

所以这里不能只补一边，而是要同时把“地址分流”和“绘图逻辑”接上。只有这样，`videotest` 才不会停留在“程序在跑，但屏幕上没变化”的状态。

NEMU 端真正把 MMIO 接进内存访问路径的关键代码在 `memory.c` 里：

```
uint32_t paddr_read(paddr_t addr, int len) {
    int map_NO = is_mmio(addr);
    if (map_NO != -1) {
        return mmio_read(addr, len, map_NO);
    }
    return pmem_rw(addr, uint32_t) & (~0u >> ((4 - len) << 3));
}

void paddr_write(paddr_t addr, int len, uint32_t data) {
    int map_NO = is_mmio(addr);
    if (map_NO != -1) {
        mmio_write(addr, len, data, map_NO);
        return;
    }
    memcpy(guest_to_host(addr), &data, len);
}
```

这一步的意义很直接：如果一个地址其实映射到了设备空间，那么后续读写就不能再落回普通物理内存 `pmem[]`，而必须转发给对应设备的回调逻辑。也正是因为有了这个分流，VGA 对应的显存映射区域才能真正“像设备一样”工作。

AM 端 `_draw_rect()` 的关键代码则是：

```
void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h) {
    int copy_w = min(w, _screen.width - x);
    if (copy_w <= 0 || x >= _screen.width || y >= _screen.height) {
        return;
    }

    for (int j = 0; j < h && y + j < _screen.height; j++) {
        memcpy(&fb[(y + j) * _screen.width + x], pixels, copy_w *
sizeof(uint32_t));
        pixels += w;
    }
}
```

这段实现里最关键的不是 `memcpy` 本身，而是“按行复制”和“边界裁剪”这两个动作：

- 按行复制保证了输入矩形中的每一行都会落到 framebuffer 中正确的位置；
- 边界裁剪保证了矩形在靠近屏幕边缘时不会越界写入。

如果没有按行推进 `pixels += w`，那么整个矩形会被错误地重复拷贝第一行；如果没有裁剪，测试程序传入靠近边界的矩形时，就很容易把后续显存或普通内存一起破坏。

这里再结合 `vga.c` 中的注册逻辑来看，会更清楚显存是如何真正接上的：

```
vmem = add_mmio_map(VMEM, 0x80000, vga_vmem_io_handler);
```

也就是说，VGA 显存对应的物理地址区间先被注册为一段 MMIO 映射；之后 CPU 对这段地址做读写时，会先在 `memory.c` 中命中 `is_mmio(addr)`，再转到 `mmio_read/mmio_write`；最后才落到具体设备的回调逻辑。这条链路如果中间任何一环没接上，`videotest` 看到的现象都会是“程序在运行，但屏幕没有正确变化”。

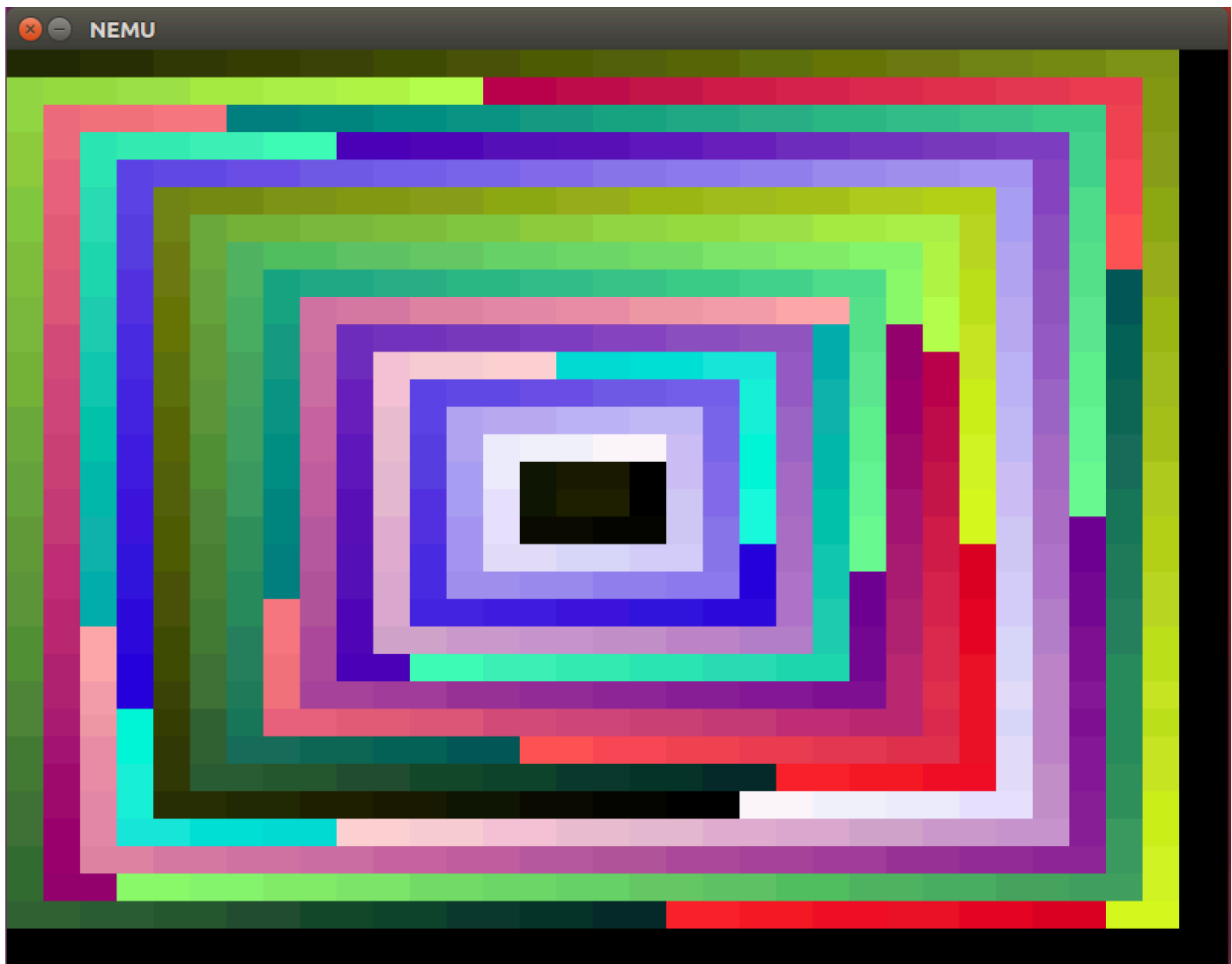
从整体结构看，VGA 这一部分其实正好把 PA2 阶段三的两种 I/O 方式都串起来了：

- 键盘、时钟、串口主要走端口映射 I/O，也就是 `pio_read/pio_write`；
- VGA 显存主要走内存映射 I/O，也就是 `mmio_read/mmio_write`。

这两种方式在代码里最终都落到同一个问题上：CPU 执行到某条 I/O 指令或者某次显存写入时，能不能被正确分流到设备模型，而不是误落回普通内存或普通寄存器语义。PA2 阶段三真正训练的，其实就是把这两套访问方式都打通。

下图对应 `videotest` 的运行界面。它说明显存 MMIO 和 `_draw_rect()` 已经能够共同工作，程序写出的颜色块可以真正显示到屏幕上。

图 6-5 *videotest* 运行结果

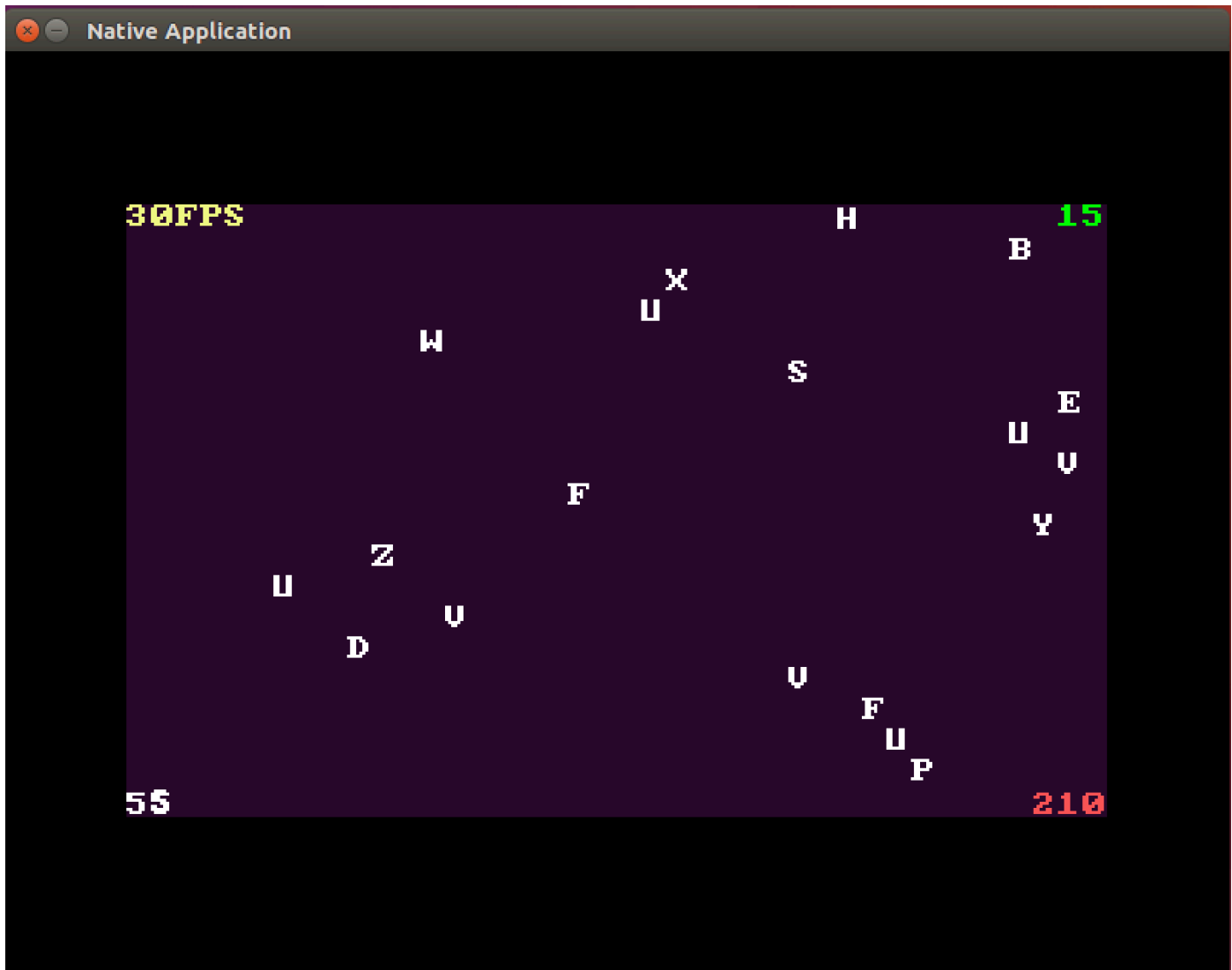


6.6 运行 *typing*

完成 VGA 和键盘后，最终可以运行 *typing*。这一步意味着程序已经能够同时使用输入和图形输出，表明 NEMU 在 PA2 阶段已经从“只会执行一小段客户代码”的模拟器，演进为具备基础交互能力的冯诺依曼计算机系统。

下图展示的是 *typing* 的运行界面，它直接反映出图形输出和键盘输入已经能够同时工作。

图 6-6 *typing* 运行结果



如果按更直观的效果来看，最终程序也能够进一步运行到小游戏界面，这说明 PA2 的输入输出链路已经基本打通。

下面两张图进一步展示了程序进入交互界面后的效果，它们主要说明阶段三实现的输入输出能力已经能够支撑更完整的图形程序运行。

不过从实际运行体验来看，这类图形化程序在当前实验环境下明显比较卡顿，画面刷新和交互响应都不够流畅。结合运行环境判断，这一现象更可能来自虚拟机本身的性能限制，而不是输入输出链路没有接通：因为程序已经能够正常显示界面、持续刷新，并且键盘输入也能生效，只是整体帧率偏低。和其他同学交流后，发现这类图形程序在虚拟机里出现卡顿也比较常见，因此这里更倾向于把它看作实验环境带来的性能瓶颈，而不是功能性错误。

图 6-7 图形程序运行界面一

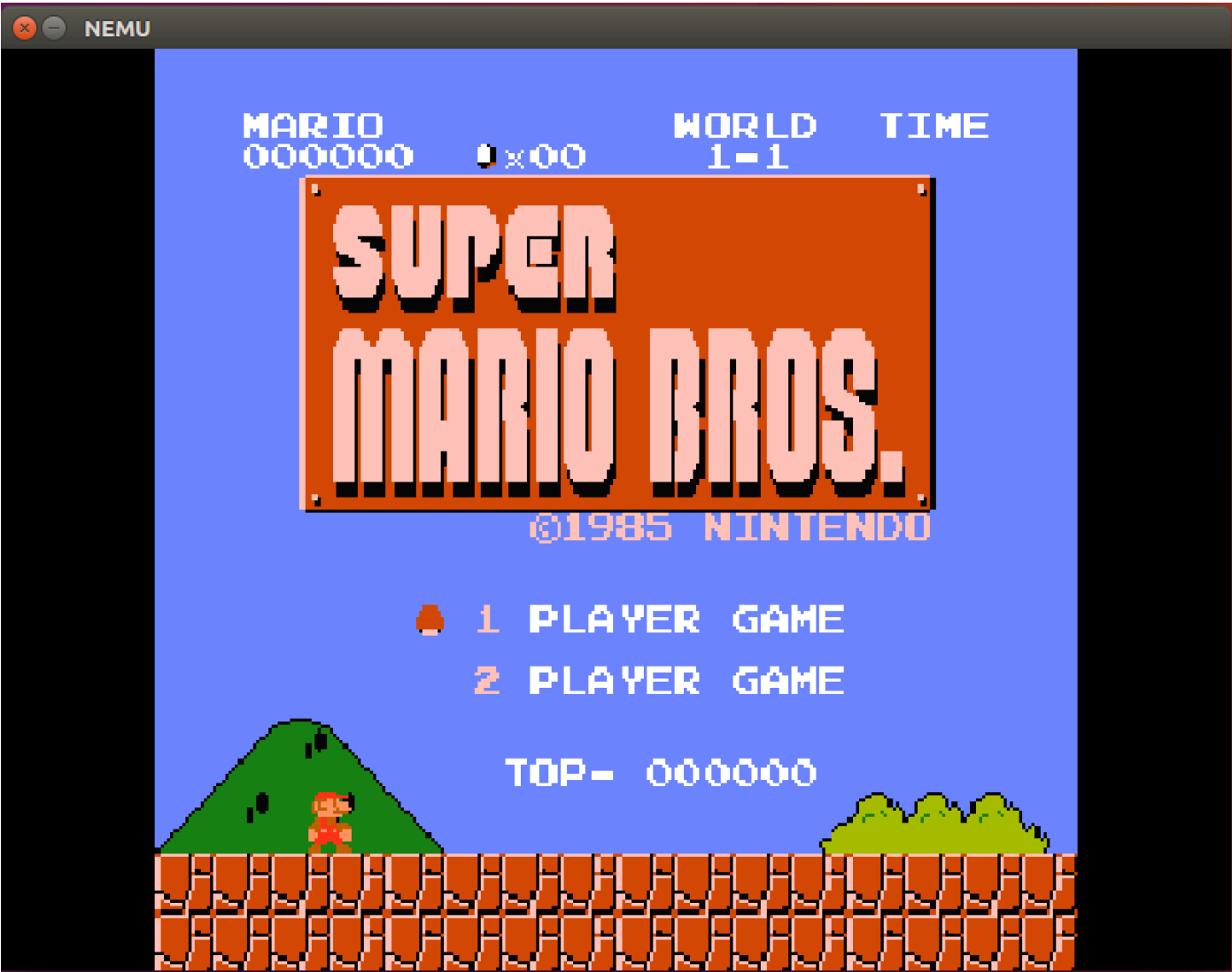


图 6-8 图形程序运行界面二



6.7 阶段三小结

阶段三最明显的感受是：PA2 到这里已经不再只是“补 CPU 指令”，而是在把一台最小可运行机器的输入、输出和运行时环境一点点接齐。串口、时钟、键盘、VGA 看起来像是分散的几个模块，但真正跑通 `typing` 之后会发现，它们其实构成了一条完整链路：

- 上层程序通过 AM 调用统一接口；
- AM 再通过端口映射或内存映射访问 NEMU 中的设备模型；
- 设备模型把这些读写转换成终端输出、时间流逝、按键事件和屏幕刷新。

也正是到了这一阶段，NEMU 才真正从“能跑一段客户代码”的模拟器，演化成了一个具备基础交互能力的冯诺依曼计算机系统。

如果回到实现细节去看，这一阶段真正打通的是几条彼此相连的链路：

- CPU 端先要能正确执行 `in/out` 这类 I/O 指令；
- NEMU 端再把这些端口访问映射到定时器、键盘、串口等设备模型；

- 对于显存这类内存映射设备，还要在 `paddr_read/paddr_write` 中额外做 MMIO 分流；
- AM 端最后把这些底层设备包装成 `_uptime()`、`_read_key()`、`_draw_rect()` 这样的统一接口。

所以阶段三虽然表面上看起来是“补设备”，实际上同时要求：

- CPU 指令层已经足够稳定；
- 物理内存访问路径已经能区分普通内存和 MMIO；
- AM 和 NEMU 之间的接口协议已经对齐。

也正因为这些环节是串在一起的，阶段三的 bug 往往不会只表现为“某一个函数没写”，而会表现为“程序能跑，但输出不对、输入没反应、画面不刷新、计时异常”等更偏系统层面的现象。

7. 必答题

7.1 `static inline` 为什么不能随便去掉

答：这一题如果只说“这样写是为了优化”其实不太够。结合 `rtl.h` 里的实际情况来看，`static` 和 `inline` 分别处理的是“链接属性”和“函数展开方式”这两个问题。

以 `rtl_li()` 为例，当前代码是：

```
static inline void rtl_li(rtlreg_t* dest, uint32_t imm) {
    *dest = imm;
}
```

`rtl.h` 是头文件，会被很多 `.c` 文件同时包含。也就是说，这个函数定义不是只出现在一个地方，而是会随着 `#include` 被展开到很多编译单元里。所以这里一旦把 `static` 和 `inline` 随便改掉，问题基本都会出在“头文件里定义函数”这件事上。

先看去掉 `static`、只保留 `inline` 的情况。这样一来，函数就变成了带外部链接属性的 `inline` 函数。头文件里虽然还有函数体，但工程里并没有再单独提供一份普通的全局定义给链接器使用。结果就是：编译阶段看起来没问题，真正到链接阶段才会发现这个符号找不到。为了验证这一点，我单独做了一个最小实验：把同一个只含 `inline` 的函数放进头文件，再让两个 `.c` 文件同时包含并调用，最后链接时得到的报错是：

```
Undefined symbols for architecture x86_64:
  "_rtl_li", referenced from:
      _fa in a.o
      _fb in b.o
ld: symbol(s) not found for architecture x86_64
```

这说明只保留 `inline` 的问题不在语法，而是在链接阶段没有可用的外部定义。

再看去掉 `inline`、只保留 `static` 的情况。这样每个包含 `rtl.h` 的编译单元都会生成一份局部的 `rtl_li()` 实体。因为它还是 `static`，所以不会和别的编译单元打架；但新的问题也马上出来了。`rtl.h` 里有很多 RTL helper，并不是每个 `.c` 文件都会真的用到每一个 helper。只要某个源文件包含了这个头文件，却没有实际调用这个函数，编译器就会把它当成“定义了但没有使用的静态函数”。而 NEMU 的 `CFLAGS` 里又开了 `-Wall -Werror`，所以这个告警会直接变成编译错误。对应的报错是：

```
error: unused function 'rtl_li' [-Werror,-Wunused-function]
```

这也说明去掉 `inline` 以后，问题不只是“少了一个优化提示”，而是真的会在当前工程配置下直接编译不过。

最后看 `static` 和 `inline` 两者都去掉的情况。此时头文件里放着的就是一个普通的全局函数。因为这个头文件会被多个 `.c` 文件包含，所以每个编译单元都会各自生成一份同名的全局符号，最后链接时就会撞在一起。对应的报错是：

```
duplicate symbol '_rtl_li' in:
  d.o
  e.o
ld: 1 duplicate symbol for architecture x86_64
```

这说明去掉两者以后，本质上就是把一个本来只适合写在单个源文件里的全局函数定义，错误地放进了会被反复展开的头文件里。

所以，`rtl.h` 里写成 `static inline` 并不是随手这么写，而是和当前工程的组织方式正好对应：

- `static` 让函数具有内部链接属性，避免头文件中的函数定义在多个编译单元之间发生符号冲突；
- `inline` 让这些非常短小的 helper 倾向于直接在调用点展开，同时也避免在未使用时产生多余的局部函数实体；

- 两者配合，才适合放在 `rtl.h` 这种会被大量源文件包含的头文件里。

7.2 在 `nemu` 目录下执行 `make` 后，`build/nemu` 是如何生成的

答：这一题我觉得分成两层来看会比较清楚：一层是 Makefile 自己是怎么组织规则的，另一层是 C 程序从源文件到可执行文件到底经历了什么。

先看 Makefile 的入口。`nemu/Makefile` 一开头先定义了几个关键变量：

```
NAME = nemu
BUILD_DIR ?= ./build
OBJ_DIR ?= $(BUILD_DIR)/obj
BINARY ?= $(BUILD_DIR)/$(NAME)
```

这几行先把目标名字、构建目录、目标文件目录和最终可执行文件路径都定下来了。代入之后，最后生成的二进制文件就是：

```
build/nemu
```

接着 Makefile 通过：

```
include Makefile.git
```

把另一个 Makefile 片段包含进来。这个片段主要补充了和 `git_commit` 相关的辅助规则，所以当前这个 Makefile 并不是单独工作的，而是先读主 Makefile，再把额外规则一起拼进来解释。

然后它把默认目标设成：

```
.DEFAULT_GOAL = app
```

所以在 `nemu` 目录下直接敲 `make`，实际上就等价于执行 `make app`。而 `app` 规则又写成：

```
app: $(BINARY)
```

也就是说，`make` 最后真正要做的事情，就是把 `$(BINARY)` 也就是 `build/nemu` 生成出来。

为了做到这一步，`Makefile` 先用变量和函数把所有源文件列出来：

```
SRCS = $(shell find src/ -name "*.c")
OBJS = $(SRCS:src/%.c=$(OBJ_DIR)/%.o)
```

这里 `$(shell ...)` 会先调用 `shell` 命令，把 `src/` 目录下所有 `.c` 文件找出来；然后再用模式替换，把每个源文件路径映射成对应的目标文件路径。例如：

```
src/cpu/exec/control.c
```

会被映射成：

```
build/obj/cpu/exec/control.o
```

这一步其实就是在做批量展开和路径映射。

接下来真正决定“`.c` 怎么变成 `.o`”的是这条模式规则：

```
$(OBJ_DIR)/%.o: src/%.c
    @echo + CC $<
    @mkdir -p $(dir $@)
    @$(CC) $(CFLAGS) -c -o $@ $<
```

这里可以拆开来看：

- `$(OBJ_DIR)/%.o: src/%.c` 表示任意一个目标文件都依赖于相同相对路径下的源文件；
- `$<` 表示当前规则的第一个依赖文件，也就是对应的 `.c`；
- `$@` 表示当前规则的目标文件，也就是对应的 `.o`；
- `mkdir -p $(dir $@)` 用来保证目标文件所在目录先被创建出来；
- `$(CC) $(CFLAGS) -c -o $@ $<` 则是真正的编译命令，只编译、不链接。

这里虽然 Make 自己本身就带有 implicit rules，但当前 Makefile 又显式写了一条更适合本工程目录结构的 pattern rule，相当于按照自己的目录布局把“.c 编译成 .o”这件事重新定义了一遍。

为了确认 Make 实际会执行哪些命令，我又用 `make -n` 看了一次构建过程。输出里可以直接看到，Make 会先依次执行大量形如下面的命令：

```
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o
build/obj//cpu/exec/control.o src//cpu/exec/control.c
```

这说明在编译阶段，每个 .c 文件都会单独编译成一个 .o 文件。这里的 `-I./include` 用来告诉编译器头文件搜索路径，`-c` 表示只编译不链接，`-ggdb` 用来保留调试信息，`-O2` 用来做优化，而 `-Wall -Werror` 则保证潜在问题不会被轻易放过去。

头文件虽然不会像 .c 一样直接编译成 .o，但也不是完全不参与构建。这一点体现在：

```
CFLAGS += -O2 -MMD -Wall -Werror -ggdb $(INCLUDES)
#include $(OBSJ:.o=.d)
```

其中 `-MMD` 会让编译器在生成每个 .o 的同时，再生成一个对应的 .d 依赖文件，里面记录这个 .o 依赖了哪些 .h。后面的 `-include $(OBSJ:.o=.d)` 会把这些依赖文件重新包含进 Makefile。这样一来，只要某个头文件改了，下一次执行 `make` 时，所有依赖它的 .o 都会自动重新编译。这就是 .h 文件参与构建的方式。

等所有 .o 都准备好之后，Make 才会执行最后的链接规则：

```
$(BINARY): $(OBSJ)
@echo + LD $@
@$(LD) -O2 -o $@ $^ -lSDL2 -lreadline
```

这里 `$^` 表示全部依赖，也就是前面列出来的所有目标文件。链接器把这些 .o 合并起来，再链接上 `SDL2` 和 `readline` 库，最后生成 `build/nemu`。这一阶段和前面的编译阶段不同，前面处理的是单个源文件，这里处理的是整个工程。

所以，在 `nemu` 目录下敲入 `make` 后，整个过程可以概括为：

1. 读取主 Makefile，并包含 `Makefile.git`。
2. 以 `app` 作为默认目标，进一步展开到最终目标 `build/nemu`。
3. 用 `find` 枚举 `src/` 下所有 .c，再映射成对应的 .o 路径。

4. 按 pattern rule 把每个 `.c` 单独编译成 `.o`，同时借助 `-MMD` 记录头文件依赖。
5. 当所有 `.o` 都齐全后，把它们统一链接成 `build/nemu`。

这样看起来，Makefile 负责的是“决定哪些文件先做、哪些文件后做、每一步具体怎么做”；而编译和链接负责的是“把源代码一步步变成最后可以执行的二进制文件”。两部分合起来，最后就得到了 `nemu/build/nemu`。

8. 思考

8.1 一条指令在 NEMU 中的执行过程

结合本次实验，一条指令在 NEMU 中的执行过程可以概括为：

1. 当前 `eip` 所指位置取出指令字节。
2. 解释首字节为 `opcode`。
3. 在 `opcode_table` 中查找对应的译码和执行逻辑。
4. 使用 decode helper 获取源操作数和目的操作数信息。
5. 使用 exec helper 通过 RTL 指令完成真实语义。
6. 最后统一更新 `eip`，进入下一条动态指令。

这一点在当前 `exec.c` 里体现得很直接。执行框架的关键代码如下：

```
static inline void idex(vaddr_t *eip, opcode_entry *e) {
    if (e->decode)
        e->decode(eip);
    e->execute(eip);
}

make_EHelper(real) {
    uint32_t opcode = instr_fetch(eip, 1);
    decoding.opcode = opcode;
    set_width(opcode_table[opcode].width);
    idex(eip, &opcode_table[opcode]);
}

static inline void update_eip(void) {
    cpu.eip = (decoding.is_jump ? (decoding.is_jump = 0, decoding.jump_eip) :
    decoding.seq_eip);
}
```

这几段代码正好把“取指、查表、译码、执行、更新 `eip`”串成了一条完整链路：

- `instr_fetch()` 负责按当前 `eip` 取出 opcode;
- `opcode_table[opcode]` 负责把 opcode 映射到对应的 decode / exec helper;
- `idex()` 先做译码, 再做执行;
- `update_eip()` 最后统一处理顺序执行还是跳转执行。

这也是为什么阶段一里实现 `call`、`jmp`、`jcc` 时, 并不直接在 helper 里硬改 `cpu.eip`, 而是通过设置 `decoding.is_jmp` 和 `decoding.jmp_eip`, 让执行流继续保持统一。

8.2 手册的重要性

PA2 里有几个地方都很能说明 i386 手册为什么必须查, 而不能完全凭感觉写:

- `call rel32` 中 `rel32` 是有符号偏移;
- `push imm8` 需要先符号扩展;
- `inc/dec` 和 `sub` 对标志位影响不同;
- `EFLAGS` 的初值不能随意设置。

这些细节如果判断错了, 后面的实现即使表面上看起来像是对的, 实际运行也会偏掉。也正因为这些内容在手册里写得非常清楚, 所以 PA2 的关键能力之一并不是记住所有指令, 而是能快速查到当前这个问题该读手册的哪一段。

再往下看当前实现, 也能发现很多地方其实就是把手册里的语义逐条翻译成代码。例如 `decode_op_SI()` 里专门写了:

```
uint32_t imm = instr_fetch(eip, op->width);
if (op->width == 1) {
    op->simm = (int8_t)imm;
}
else {
    op->simm = (int32_t)imm;
}
```

这就是把“sign immediate”这个手册概念直接落到了译码阶段。如果没有先在手册里确认 `call rel32`、`push imm8` 这一类操作数的符号属性, 光看机器码很容易误判成普通立即数, 从而让后面的执行语义整体偏掉。

9. 实验过程中遇到的问题与解决方法

这一部分单独整理了 PA2 中最典型、也最能反映调试过程的几个问题。前面的阶段一到阶段三已经分别说明了实现思路，这里更侧重记录问题是怎么暴露出来的、当时是怎么判断原因的、最后又是如何修掉的。

9.1 `invalid opcode` 的第一反应不应该是猜，而是先去反汇编里定位机器码

PA2 里最频繁出现的问题就是 `invalid opcode`。刚开始最自然的反应是去猜“是不是上一条指令实现错了”，但这样效率其实很低。后来逐渐形成了一个更稳定的做法：

1. 先看报错给出的 `eip`；
2. 再看当前机器码前几个字节；
3. 到反汇编文件里定位这一条具体指令；
4. 最后再决定它是“未实现指令”，还是“已有指令实现错了”。

例如第一次 `dummy` 报错时：

```
invalid opcode(eip = 0x0010000a): e8 01 00 00 00 ...
```

再去 `dummy-x86-nemu.txt` 里定位，就能直接看到这是：

```
call 100010 <_trm_init>
```

这时问题就很清楚了：不是整条执行链都坏了，而是 `0xe8` 对应的 `call rel32` 还没有接进去。后面排查 `rol`、`imul` 这些问题时，这套方法也一样有效。

9.2 `call rel32` 最容易写错的地方不是压栈，而是把相对偏移当成无符号数

`call rel32` 真正实现时，看上去好像只要“把返回地址压栈，再改 `eip`”就够了，但真正踩坑之后才发现，最关键的问题其实是 `rel32` 的符号属性。

如果把 `rel32` 当成普通无符号立即数去读，短期内有些向前跳的例子可能碰巧还能跑，但一旦遇到负偏移控制流，跳转目标就会完全错掉。这个问题最后是通过在 `decode_op_SI()` 中显式做符号扩展解决的。也正是这次之后，对“先查手册确认立即数到底是 signed 还是 unsigned”这件事印象特别深。

9.3 group 指令最容易犯的错误不是语义写错，而是重复译码

PA2 后面补 **sub**、移位类指令时，遇到的一个很典型问题是：明明 helper 已经写了，运行时结果还是不对。继续查下去才发现，问题并不在执行语义，而在 group 指令的接法。

像 **83 / 5** 这样的指令，外层 opcode 已经把操作数译码好了，所以进入 group 表之后，应该直接写：

```
EX(sub)
```

而不是再套一层 **IDEX(...)**。如果在 group 表里重复译码，操作数状态会被破坏，看起来就像“指令实现逻辑有 bug”，实际上是译码阶段已经错了。

9.4 difftest 的第一处差异出现在 **eflags**，并不一定意味着整条执行语义都错了

开启 **DIFF_TEST** 后，第一次看到：

```
Difftest mismatch at eip = 0x00100000: eflags, NEMU = 0x00000002, QEMU = 0x00000006
```

当时最直接的感觉是“是不是刚补的指令把标志位全搞错了”。但继续分析后发现，多出来的是 **PF**，而当前 PA2 阶段真正实现并关心的只是 **CF/ZF/SF/IF/OF**。

也就是说，这里并不是“CPU 已经严重跑偏”，而是比较策略本身太宽了。把未实现的标志位也强行纳入比较之后，difftest 一开始就会被这些无关差异淹没。后面改成 mask 之后，才真正把 difftest 调整回“帮助暴露当前阶段真实 bug”的状态。

9.5 **volatile** 这个问题，只有看过优化前后的汇编，才真正能理解它为什么和设备寄存器有关

volatile 在书上看起来很抽象，但做完那个小实验之后，理解就具体了很多。保留 **volatile** 时，编译器会在循环里反复读取内存地址；去掉之后，**-O2** 直接把代码优化

成了不会再重新读取设备寄存器的死循环。

这个实验给我的直观感受是：设备寄存器和普通内存最大的区别之一，就是“每次读取都可能有意义”。如果不告诉编译器这一点，它就会按普通变量的思路去做缓存、合并和消除，最后程序和设备之间的交互语义就会被彻底破坏。

9.6 VGA 部分最容易出现“代码两边都看着像对了，但画面就是不出来”的情况

键盘部分的问题比较直白，因为 `_read_key()` 一开始就是空实现；VGA 则更迷惑一些，因为设备代码、显存映射、上层接口表面上看都已经存在了，但运行 `videotest` 时画面还是不对。

后面顺着链路拆开看才意识到，这里不能只修一个点，而要同时保证：

- `paddr_read()` / `paddr_write()` 真正把 VGA 地址段分流到了 MMIO；
- `_draw_rect()` 真的在按矩形语义写 framebuffer，而不是还停留在占位实现。

这个问题也让我更清楚地意识到，PA2 后半段很多 bug 已经不是“单个函数写错”，而是跨模块链路没有完全接通。只有把整条链从上层应用、AM、NEMU 设备模型一路串起来看，问题才会真正清晰。

10. 总结

PA2 是一次非常完整的“从最小执行路径走向完整系统”的训练过程。我先从 `dummy` 入手，实现最基础的一批指令，再逐步补齐更多算术、逻辑、移位和控制转移指令，并在此过程中完善了 EFLAGS、RTL helper 和 `difftest` 等关键基础设施。随后又继续完成了串口、时钟、键盘、VGA 等输入输出相关内容，最终让系统运行到了 `hello`、`timetest`、`keytest`、`videotest` 和 `typing`。

和 PA1 相比，PA2 更能体现“计算机系统是如何一步一步长出来的”。它不仅让我更清楚地理解了取指、译码、执行和更新 `eip` 的全过程，也让我真正体验到：当手册、测试、反汇编、RTL 和设备模型这些要素被串联起来以后，一个原本极其简单的模拟器，是可以逐步演化成具备运行时环境和输入输出能力的完整系统的。